MEDICAL ROBOTICS - PROJECT REPORT

Marko Kuzmanoski

University of Coimbra, DEEC

Abstract

This project explores the design and implementation of control and trajectory planning strategies for planar robotic manipulators specifically tailored for such surgical applications. By integrating geometric, kinematic, and dynamic models, the study addresses critical challenges in achieving safe and precise robotic operations. The robotic system considered is a two-link planar manipulator, idealized for operations in confined surgical spaces. This work begins with the derivation of forward kinematics, enabling the mapping of joint configurations to the Cartesian workspace. The Jacobian matrices, representing the relationship between joint velocities and end-effector motion, were also computed, forming the foundation for trajectory tracking and control.

Dynamic modeling plays a crucial role in understanding the forces and torques involved in the manipulator's motion. Using the Lagrangian formulation, the project successfully derives expressions for the inertia matrix, Coriolis forces, and gravity vector. These components are vital for implementing advanced control strategies that ensure the robot's stability and responsiveness during delicate surgical tasks.

Control schemes were designed to address various operational requirements, including trajectory tracking, compliance, and force application. Strategies such as proportional-derivative (PD) control, Cartesian force application with gravity compensation, and compliance control were tested. Each approach was evaluated in terms of its ability to handle specific scenarios encountered in surgical environments, such as maintaining tool precision or responding to external forces.

A significant aspect of the project is the development of a MATLAB/Simulink-based simulation framework. This simulator visualizes the robot's behavior under different control laws and trajectory configurations, providing insights into the system's performance. Elliptical and diagonal trajectories were generated and tested, highlighting the manipulator's adaptability to various path-planning requirements.

This report details the methodologies and results of the project, beginning with the kinematic and dynamic modeling of the planar manipulator. The project extends to the implementation of control strategies and their testing under simulated conditions. By integrating robust theoretical models with practical simulations, the study aims to advance the capabilities of robotic-assisted surgical systems, ensuring precision, safety, and adaptability in complex medical environments.

All the MATLAB codes, functions, control schemes, and simulation frameworks developed during the project are included in the full documentation. Additionally, videos demonstrating the results of key simulations are attached, offering a dynamic and visual representation of the system designs and experiments.

1 Forward Kinematics

Forward kinematics is a fundamental concept in robotics that determines the position of a robot's end-effector in Cartesian coordinates based on the given joint angles and link lengths. For the two-link planar robotic manipulator used in this study, the forward kinematics equations are as follows:

$$\begin{aligned} x &= l_1 \cos(q_1) + l_2 \cos(q_1 + q_2) \\ y &= l_1 \sin(q_1) + l_2 \sin(q_1 + q_2) \end{aligned} \tag{1}$$

Here:

- q_1 and q_2 are the joint angles of the manipulator.
- l_1 and l_2 represent the lengths of the first and second links, respectively.
- x and y are the Cartesian coordinates of the end-effector.

1.1 Robot Configuration

The two-link manipulator is illustrated in Figure 1. The diagram highlights the robot's geometry, including the joint angles q_1, q_2, link lengths l_1, l_2, and the resulting end-effector position (x, y).

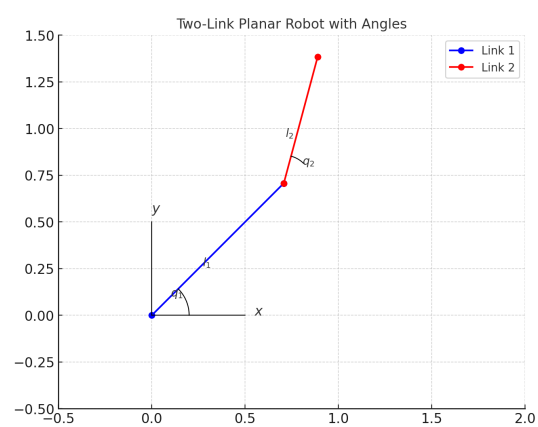

Figure 1: Two-link planar manipulator showing joint angles q_1, q_2 and link lengths l_1, l_2.

1.2 Significance in Robotic Surgery

Forward kinematics is crucial in robotic-assisted minimally invasive surgeries, as it allows:

- Precise mapping of joint angles to end-effector positions, ensuring accurate tool placement.
- Development of trajectory planning algorithms that enable safe navigation around sensitive anatomical structures.
- Integration with control systems for real-time adjustment of the robot's movements during procedures.

1.3 Implementation and Analysis

To compute the forward kinematics:

1. The joint angles q_1, q_2 were defined based on the desired configuration.

2. The link lengths l_1, l_2 provided by the robotic system design were used.

3. The equations were applied to calculate the Cartesian position of the end-effector.

The forward kinematics model was implemented and served as the foundation for trajectory generation and control. This enabled precise and efficient robotic movements, meeting the requirements of minimally invasive surgical tasks.

2 Jacobian Computation for the Planar Robot

In robotic systems, Jacobian matrices are essential tools for understanding the relationship between joint velocities and the linear and angular velocities of various points on the robot, such as the end-effector and the centers of mass. For the two-link planar robot in this project, Jacobians were computed for both the end-effector and the centers of mass for linear and angular motions.

2.1 Jacobian for the End-Effector

The Jacobian matrix for the end-effector defines the relationship between the joint velocities and the linear velocities of the end-effector in Cartesian space. Using the robot's kinematic equations, the Jacobian matrix was derived as:

$$J_{\text{end-effector}} = \begin{bmatrix} -l_1 \sin(q_1) - l_2 \sin(q_1 + q_2) & -l_2 \sin(q_1 + q_2) \\ l_1 \cos(q_1) + l_2 \cos(q_1 + q_2) & l_2 \cos(q_1 + q_2) \end{bmatrix} \tag{2}$$

This matrix maps the joint velocities $\dot{q}_1$ and $\dot{q}_2$ to the Cartesian velocities of the end-effector. Its computation is critical for trajectory tracking and motion control, enabling precise manipulations during robotic-assisted surgeries.

2.2 Jacobian for the Centers of Mass

To further analyze the dynamics of the robot, Jacobians for the centers of mass of both links were computed. These include:

- **Linear velocity Jacobians:** These matrices relate joint velocities to the linear velocities at the centers of mass of each link.

- **Angular velocity Jacobians:** These matrices provide the angular velocity information for each link.

The linear velocity Jacobians were derived as:

$$J_{v,c1} = \begin{bmatrix} -l_{c1} \sin(q_1) & 0 \\ l_{c1} \cos(q_1) & 0 \\ 0 & 0 \end{bmatrix} \tag{3}$$

and

$$J_{v,c2} = \begin{bmatrix} -l_1 \sin(q_1) - l_{c2} \sin(q_1 + q_2) & -l_{c2} \sin(q_1 + q_2) \\ l_1 \cos(q_1) + l_{c2} \cos(q_1 + q_2) & l_{c2} \cos(q_1 + q_2) \\ 0 & 0 \end{bmatrix} \tag{4}$$

The angular velocity Jacobians for both links were:

$$J_{w,1} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \quad J_{w,2} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}. \tag{5}$$

These Jacobians provide crucial insights into the dynamics of the robot's links, facilitating the implementation of advanced control strategies.

The computation of these Jacobians was pivotal in the development of control and trajectory planning algorithms. The end-effector Jacobian served as the foundation for tasks requiring precise positioning and trajectory tracking. The Jacobians for the centers of mass were essential for dynamic modeling, ensuring accurate force and torque calculations for each link.

The computed Jacobians were validated through simulations and integrated into the control and trajectory planning modules. These simulations demonstrated the robot's ability to execute complex movements with high precision. Videos of these simulations, included in the project documentation, illustrate the robot's dynamic responses under various control strategies.

3 Robotic Dynamics Computation Using Lagrangian Formulation

This section details the derivation and MATLAB implementation of the inertia matrix $M(q)$, Coriolis matrix $C(q, \dot{q})$, and gravity vector $g(q)$ for the planar robot described in the project. These components are integral to the dynamic model, which is expressed as:

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) = \tau \tag{6}$$

where τ represents the applied joint torques. The derivation leverages the Lagrangian formulation and symbolic computations to ensure accuracy and consistency.

3.1 Inertia Matrix $M(q)$

The inertia matrix $M(q)$ captures the robot's mass and its distribution, influencing its acceleration. The kinetic energy of the system contributes to the elements of $M(q)$, which are expressed as:

$$M(q) = \begin{bmatrix} M_{11} & M_{12} \\ M_{21} & M_{22} \end{bmatrix} \tag{7}$$

with the individual elements given by:

$$M_{11} = m_2 l_1^2 + 2m_2 \cos(q_2) l_1 l_{c2} + m_1 l_{c1}^2 + m_2 l_{c2}^2 + I_{1,czz} + I_{2,czz},$$
$$M_{12} = M_{21} = m_2 l_{c2}^2 + m_2 \cos(q_2) l_1 l_{c2} + I_{2,czz},$$
$$M_{22} = m_2 l_{c2}^2 + I_{2,czz}.$$

In this formulation:

- m_1, m_2: Masses of the links,

- l_1, l_2: Link lengths,

- l_{c1}, l_{c2}: Distances to the center of mass,

- $I_{1,czz}, I_{2,czz}$: Moments of inertia about the center of mass.

The implementation of this matrix is provided in *InertiaMatrix.m*, ensuring adaptability to different system parameters.

3.2 Coriolis Matrix $C(q, \dot{q})$

The Coriolis matrix $C(q, \dot{q})$ accounts for velocity-dependent forces arising from the interaction between the links during motion. The elements are derived from the Christoffel symbols of the first kind and are expressed as:

$$C(q, \dot{q}) = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}, \tag{8}$$

where

$$\begin{aligned} C_{11} &= \dot{q}_1 l_1 l_{c2} m_2 \sin(q_2) - \frac{3}{2}\dot{q}_2 l_1 l_{c2} m_2 \sin(q_2), \\ C_{12} &= \frac{1}{2}\dot{q}_1 l_1 l_{c2} m_2 \sin(q_2) - \dot{q}_2 l_1 l_{c2} m_2 \sin(q_2), \\ C_{21} &= \dot{q}_1 l_1 l_{c2} m_2 \sin(q_2) - \frac{1}{2}\dot{q}_2 l_1 l_{c2} m_2 \sin(q_2), \\ C_{22} &= \frac{1}{2}\dot{q}_1 l_1 l_{c2} m_2 \sin(q_2). \end{aligned}$$

Here, $\dot{q}_1, \dot{q}_2$ are the joint velocities, and $\sin(q_2)$ captures the coupling effects between links. This matrix is implemented in *CoriolisMatrix.m*.

3.3 Gravity Vector $g(q)$

The gravity vector $g(q)$ represents the forces due to gravity acting on the robot's links. It is crucial for maintaining stability and ensuring accurate control under the influence of gravity. The vector is expressed as:

$$g(q) = \begin{bmatrix} G_1 \\ G_2 \end{bmatrix}, \tag{9}$$

where

$$\begin{aligned} G_1 &= g m_2 (l_{c2} \sin(q_1 + q_2) + l_1 \sin(q_1)) + g m_1 l_{c1} \sin(q_1), \\ G_2 &= g m_2 l_{c2} \sin(q_1 + q_2). \end{aligned}$$

Here:

- g: Acceleration due to gravity,
- q_1, q_2: Joint angles of the robot.

The implementation of this computation is contained in *GravityVector.m*.

The detailed derivation and implementation of $M(q)$, $C(q, \dot{q})$, and $g(q)$ provide a robust framework for modeling and control of the robotic system. The MATLAB implementations ensure flexibility and accuracy for further computational tasks in this project. The functions are structured as follows:

- **Inertia Matrix**: *InertiaMatrix.m*
- **Coriolis Matrix**: *CoriolisMatrix.m*
- **Gravity Vector**: *GravityVector.m*

These implementations serve as a foundation for the advanced control and simulation tasks outlined in the project goals.

4 Simulation of Robot Dynamics with Simple Initial Conditions

This section presents the development of a robot simulator based on MATLAB functions and Simulink blocks. The simulator was tested with prescribed initial conditions, zero control and external torques, and varying friction terms. Observations were made regarding the robot's dynamic response under these scenarios.

4.1 Mathematical Formulation

The robot's dynamics are governed by the equation of motion:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}) + \mathbf{F_v}\dot{\mathbf{q}} = \boldsymbol{\tau}, \tag{10}$$

where:

- $\mathbf{M}(\mathbf{q})$: Inertia matrix.
- $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$: Coriolis and centripetal forces matrix.
- $\mathbf{G}(\mathbf{q})$: Gravitational forces vector.
- $\mathbf{F_v}$: Viscous friction matrix.
- $\boldsymbol{\tau}$: Generalized torque vector, where $\boldsymbol{\tau} = \boldsymbol{\tau}_c + \boldsymbol{\tau}_e$ represents the sum of control torques and external torques.

In the absence of control and external torques ($\boldsymbol{\tau}_c = \boldsymbol{\tau}_e = 0$), the joint accelerations are given by:

$$\ddot{\mathbf{q}} = \mathbf{M}(\mathbf{q})^{-1} \left(-\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} - \mathbf{G}(\mathbf{q}) - \mathbf{F_v}\dot{\mathbf{q}}\right). \tag{11}$$

4.2 Simulation Setup

The simulation was implemented using a Simulink model (`task4.slx`) and MATLAB functions. The initial conditions specified for the task were:

$$\mathbf{q}_0 = \begin{bmatrix} \pi/2 \\ \pi/4 \end{bmatrix}, \tag{12}$$

$$\dot{\mathbf{q}}_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}. \tag{13}$$

The following MATLAB functions were integrated into the Simulink model:

- `InertiaMatrix.m`: Computes $\mathbf{M}(\mathbf{q})$.
- `CoriolisMatrix.m`: Computes $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$.
- `GravityVector.m`: Computes $\mathbf{G}(\mathbf{q})$.
- `Jacobian_ee.m`: Computes the Jacobian matrix.
- `computeAcceleration.m`: Computes $\ddot{\mathbf{q}}$ for given inputs.

4.3 Simulation with $\boldsymbol{\tau}_c = \boldsymbol{\tau}_e = 0$

The simulation was initially conducted with $\boldsymbol{\tau}_c = \boldsymbol{\tau}_e = 0$, revealing that the robot exhibited free-fall behavior dominated by gravity. The joint angles $\mathbf{q}$ adjusted dynamically under gravitational forces, and the system, without any control input, did not stabilize but continued to move toward configurations dictated by gravity and inertia.

Further testing explored the effects of varying initial conditions and friction terms. Increased viscous friction ($\mathbf{F_v}$) produced dampened motion and a slower response, whereas reduced friction resulted in pronounced transient oscillations before reaching a steady state. Different initial configurations $\mathbf{q}_0$ notably influenced the robot's trajectory and final configurations, highlighting the sensitivity of dynamics to starting conditions.

Figures 2, 3, and 4 illustrate the robot's behavior under three different initial conditions.

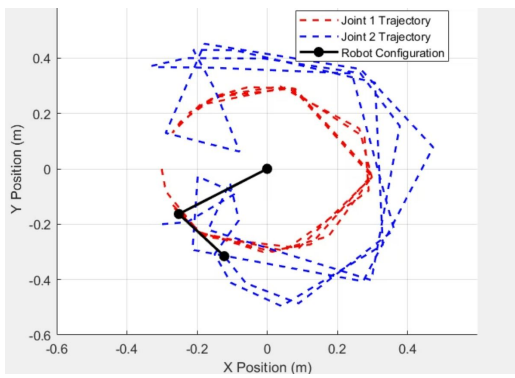

Figure 2: Robot dynamics under initial condition $\mathbf{q}_0 = \begin{bmatrix} \pi \\ \pi/2 \end{bmatrix}$.

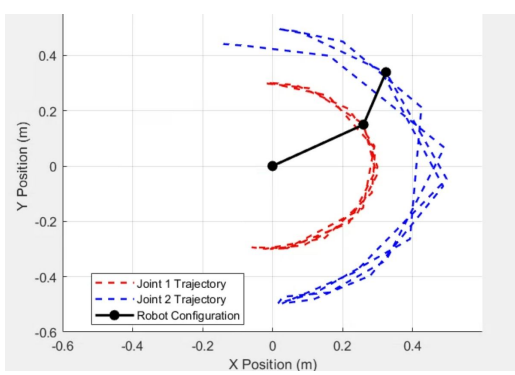

Figure 3: Robot dynamics under initial condition $\mathbf{q}_0 = \begin{bmatrix} \pi/2 \\ \pi/4 \end{bmatrix}$.

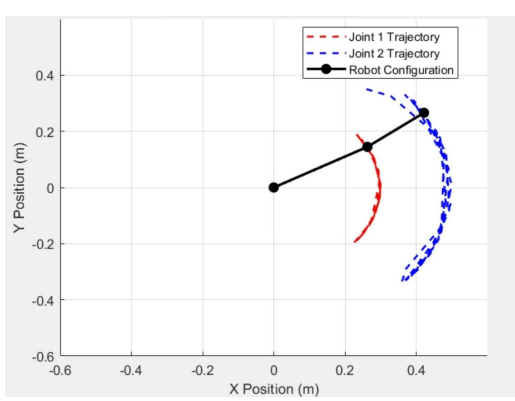

Figure 4: Robot dynamics under initial condition $\mathbf{q}_0 = \begin{bmatrix} \pi/6 \\ \pi/3 \end{bmatrix}$.

The simulations demonstrate that gravitational forces dominate in the absence of external or control torques, resulting in predictable but uncontrolled motion. The role of friction is pivotal in stabilizing the system and mitigating oscillations. Additionally, the robot's behavior is highly sensitive to initial joint configurations, affecting both transient dynamics and steady-state performance.

4.4 Simulation with $\boldsymbol{\tau} = \mathbf{G}(\mathbf{q})$

The robot was simulated with the torque input set to $\boldsymbol{\tau} = \mathbf{G}(\mathbf{q})$, representing the gravity compensation torque. Under this condition, the gravitational forces are fully compensated, leaving only inertial, Coriolis, and friction forces in the dynamics. Since the system starts with zero initial velocities ($\dot{\mathbf{q}}_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$), and no external disturbances are applied, the accelerations $\ddot{\mathbf{q}}$ remain zero throughout the simulation. Consequently, the joint angles $\mathbf{q}$ also do not change over time, and the robot remains stationary in its initial configuration.

The robot's lack of motion is expected, as the applied torques exactly counteract the gravitational forces. Figure 5 shows the robot's configuration, which remains unchanged throughout the simulation.

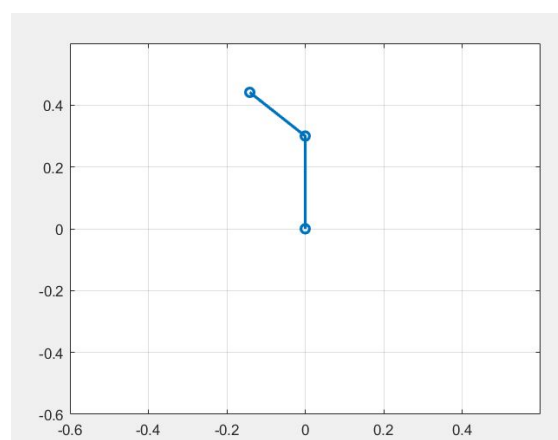

Figure 5: Robot configuration under $\boldsymbol{\tau} = \mathbf{G}(\mathbf{q})$. The robot remains stationary.

This simulation validates that the torque $\boldsymbol{\tau} = \mathbf{G}(\mathbf{q})$ effectively balances the gravitational forces, maintaining the robot's equilibrium. The stationary behavior aligns with the expected theoretical outcome, as no net forces or torques are acting to move the robot. This result demonstrates the correctness of the gravity compensation model implemented in the system.

5 Control Law Implementation and Analysis

This section describes the implementation and analysis of a control law to apply Cartesian forces at the robot's end-effector with gravity compensation. Three specific force vectors were tested:

- $\mathbf{F}_c = [0, -10]^T$
- $\mathbf{F}_c = [10, 0]^T$
- $\mathbf{F}_c = [10, -10]^T$

The relationship between the final robot position and the applied force was analyzed, with observations made on stabilization time and position errors.

5.1 Simulation Overview

The analysis involved the following steps:

1. **Gravity Compensation:** The gravity vector $\mathbf{G}(\mathbf{q})$ was determined using the `GravityVector` function, which accounts for the gravitational forces acting on the robot's joints.

2. **Transformation of Cartesian Forces:** The applied Cartesian force $\mathbf{F}_c$ was mapped to joint torques $\boldsymbol{\tau}_c$ through the transpose of the Jacobian matrix, J^T, implemented via the `controlCartesian` function.

3. **Control Torque Calculation:** The joint torques were obtained by combining the gravity compensation torques with the Cartesian force-derived torques, represented by $\boldsymbol{\tau} = \mathbf{G} + J^T\mathbf{F}_c$.

4. **Simulated Behavior:** The robot's response to each force vector was simulated within a Simulink model, documenting the resulting end-effector positions, trajectories, and stabilization times.

5. **Trajectory Visualization:** The recorded trajectories and final positions were analyzed by comparing the actual versus desired end-effector locations.

5.2 Observations and Results

The robot's response to each applied force was recorded as follows:

- For $\mathbf{F}_c = [0, -10]^T$, the robot stabilized promptly at a new position directly below the initial location. (See Fig. 6)

- For $\mathbf{F}_c = [10, 0]^T$, the robot moved horizontally to the right and stabilized more quickly than with the other force vectors. (See Fig. 7)

- For $\mathbf{F}_c = [10, -10]^T$, stabilization required a longer time due to the interplay of horizontal and vertical components. (See Fig. 8)

The stabilization time varied depending on the direction of the applied forces, with forces along diagonal directions requiring more time due to the coupled multi-axis dynamics.

The experiment demonstrated that the robot's final position aligns with the magnitude and direction of the applied Cartesian force. Key findings include:

- Forces applied along principal axes lead to quicker stabilization.

- Diagonal force vectors result in increased stabilization times due to the coupled dynamics in multiple directions.

6 Design, Implementation, and Testing of Position Control Schemes in Task Space

The primary objective of this task was to design and test P and PD control schemes to regulate the end-effector's position in Cartesian task space for a two-link planar robot. The control schemes aimed to follow a predefined elliptical trajectory, essential for precise operations in applications such as robotic-assisted surgeries. The

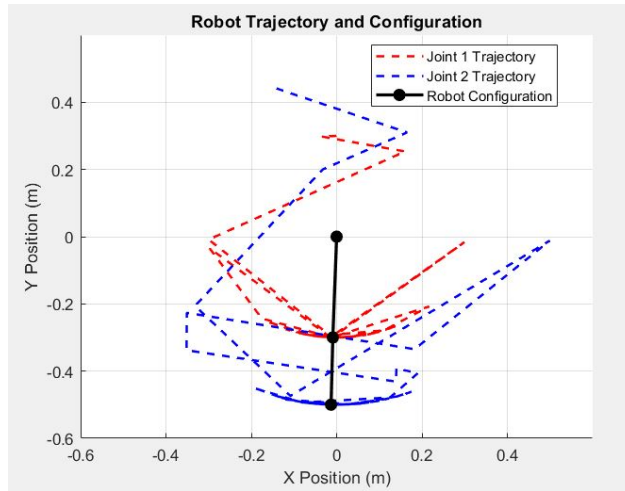

Figure 6: Trajectory for $\mathbf{F}_c = [0, -10]^T$

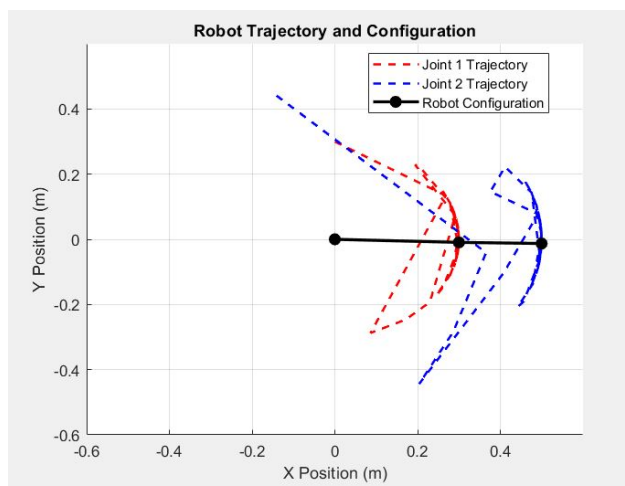

Figure 7: Trajectory for $\mathbf{F}_c = [10, 0]^T$

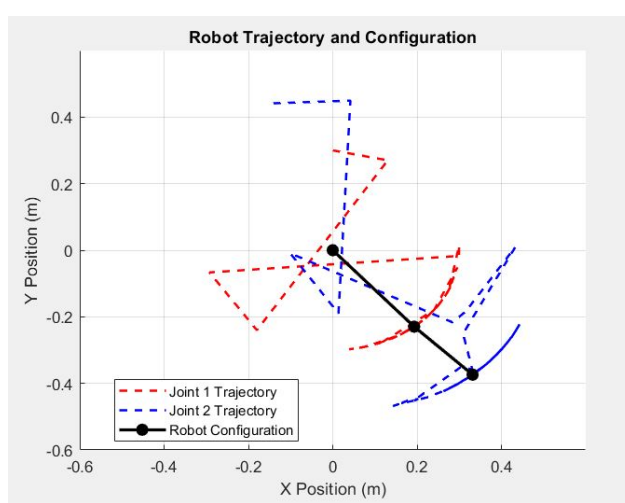

Figure 8: Trajectory for $\mathbf{F}_c = [10, -10]^T$

trajectory was centered at $(0.15, 0.2)$ with a major axis of 0.15 m along the x-axis and a minor axis of 0.1 m along the y-axis. A cyclic frequency of $1/3$ Hz was used, providing a dynamic yet controlled test scenario.

The desired trajectory was parameterized as:

$$x_d(t) = 0.15\cos(2\pi f t), \tag{14}$$

$$y_d(t) = 0.1\sin(2\pi f t), \tag{15}$$

where $f = \frac{1}{3}$ Hz.

6.1 Control Scheme Design for Elliptical Trajectory

To achieve accurate trajectory tracking, P and PD controllers were implemented in the task space. These controllers utilized feedback from the robot's sensors to compute the necessary joint torques via the Jacobian transpose method.

The control law is given by:

$$e_p = \mathbf{x}_d - \mathbf{x}_a, \tag{16}$$

$$e_v = \mathbf{v}_d - \mathbf{v}_a, \tag{17}$$

$$\tau = \mathbf{J}^T\left(\mathbf{K}_p e_p + \mathbf{K}_d e_v\right) + \mathbf{g}(\mathbf{q}), \tag{18}$$

where:

- $\mathbf{x}_d$ and $\mathbf{x}_a$ are the desired and actual end-effector positions, respectively.
- $\mathbf{v}_d$ and $\mathbf{v}_a$ represent desired and actual end-effector velocities.
- $\mathbf{J}^T$ is the Jacobian transpose, translating task space errors into joint space torques.
- $\mathbf{K}_p$ and $\mathbf{K}_d$ are diagonal proportional and derivative gain matrices.
- $\mathbf{g}(\mathbf{q})$ is the gravity compensation vector.

The gains $\mathbf{K}_p$ and $\mathbf{K}_d$ were tuned experimentally to balance precision and stability.

The implementation relied on several MATLAB functions and a Simulink scheme:

- **MATLAB Functions:**
  - `PositionControl.m`: Implements the P and PD controllers for trajectory tracking.
  - `Jacobian_ee.m`: Computes the Jacobian matrix for the end-effector.
  - `GravityVector.m`: Computes the gravity compensation vector.
  - `InertiaMatrix.m`: Calculates the mass matrix $M(q)$.
  - `CoriolisMatrix.m`: Computes Coriolis and centripetal forces.
  - `computeAcceleration.m`: Calculates joint accelerations.
  - `DisplayEllipses.m`: Visualizes the desired vs. actual trajectories.
  - `controlCartesian.m`: Converts Cartesian forces to joint torques using the Jacobian transpose.
- **Simulink Scheme:**
  - `task7.slx`: A Simulink model designed to simulate the control laws, robot dynamics, and trajectory tracking performance in a block diagram format.

6.1.1 Observations and Results

The experiments demonstrated the following key observations:

- **P Controller:** The robot followed the trajectory but exhibited noticeable steady-state errors, especially during high-curvature segments. Without velocity feedback, oscillations and delays were observed in reaching desired positions.
- **PD Controller:** Incorporating velocity feedback significantly reduced steady-state errors. The robot's trajectory tracking was smoother and more accurate. The PD controller effectively compensated for dynamic effects, particularly in transitions along the elliptical path.

Both controllers showed stable behavior under the tested conditions, but the PD controller consistently provided better precision and stability. Figure 9 illustrates the comparison between the desired and actual trajectories for both controllers. The included image in the report further highlights the enhanced performance of the PD controller.

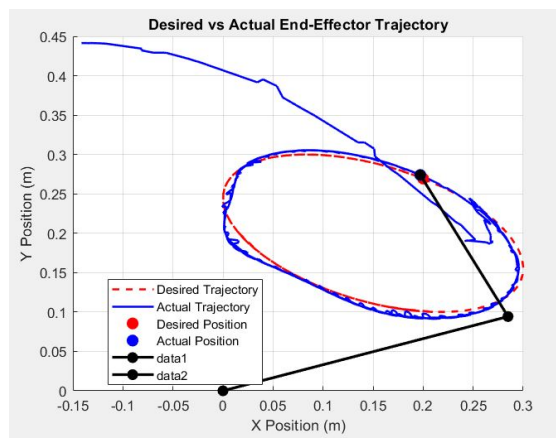

Figure 9: Comparison of desired (red) and actual (blue) trajectories for P and PD controllers.

6.1.2 Conclusions

The PD control scheme demonstrated superior performance over the P control scheme in terms of accuracy, stability, and robustness. The velocity feedback in the PD controller allowed for smoother and more precise motion, particularly in dynamic regions of the trajectory. This makes the PD controller more suitable for applications requiring high precision, such as robotic-assisted surgeries.

Additionally the task also emphasized the importance of gain tuning, as overly high gains could lead to instability or oscillations. Future work could explore:

- Adaptive control methods to dynamically adjust gains.
- Robust control to handle external disturbances and uncertainties.
- Testing more complex trajectories or introducing dynamic obstacles for further validation.

The results underscore the effectiveness of PD control for real-world robotic applications, where both precision and responsiveness are critical.

6.2 Control System Simulation for Diagonal Trajectories

The robot was also tested during diagonal trajectory tracking with arbitrary slopes to analyze the impact of varying proportional (K_p) and derivative (K_d) control gains. This evaluation focused on how different gain configurations affect the system's stability, precision, and responsiveness. The tests aimed to ensure effective end-effector control for dynamic inclined paths.

The control law applied was consistent with previous tasks, utilizing a PD controller in Cartesian space:

$$\tau = \mathbf{J}^T \left(\mathbf{K}_p e_p + \mathbf{K}_d e_v\right) + \mathbf{g}(\mathbf{q}), \tag{19}$$

where:

- $e_p = \mathbf{x}_d - \mathbf{x}_a$: Position error
- $e_v = \dot{\mathbf{x}}_d - \dot{\mathbf{x}}_a$: Velocity error
- $\mathbf{K}_p$: Proportional gain matrix
- $\mathbf{K}_d$: Derivative gain matrix
- $\mathbf{J}^T$: Jacobian transpose
- $\mathbf{g}(\mathbf{q})$: Gravity compensation

The desired diagonal trajectories were specified in task space, with arbitrary slopes providing dynamic challenges for the control scheme.

6.2.1 Observations and Results

Two sets of proportional and derivative gains were tested, and the robot's response was recorded for diagonal trajectories.

Gain Set 1: $K_p = 600$, $K_d = 200$

- The robot exhibited stable and controlled movement along the diagonal trajectory.
- Oscillations were minimal, and the end-effector closely followed the desired path.
- The system stabilized quickly, with a well-damped response ensuring precise trajectory tracking.
- This gain configuration provided smooth operation, suitable for applications requiring high precision. (See Fig. 10)

Gain Set 2: $K_p = 1000$, $K_d = 80$

- The robot displayed significant oscillations, particularly during the initial phase of motion.
- The low derivative gain relative to the proportional gain resulted in underdamped behavior, causing overshoot and slower stabilization.
- While the trajectory was eventually followed, the oscillatory behavior could pose challenges in precision-critical scenarios.
- This configuration emphasized the importance of balancing proportional and derivative gains. (See Fig. 11)

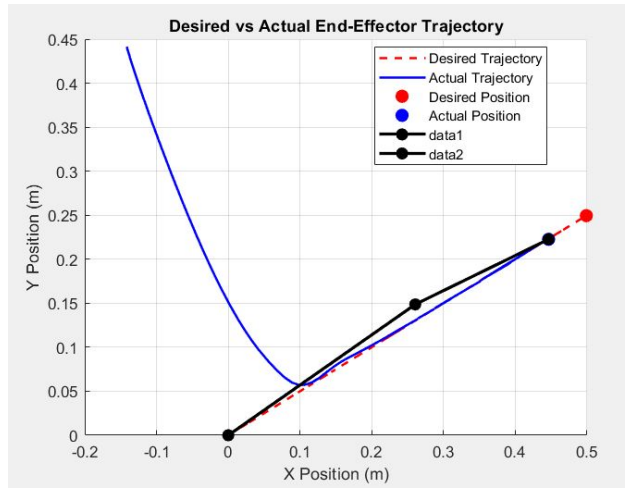

Figure 10: Trajectory tracking with $K_p = 600$, $K_d = 200$ (controlled and smooth).

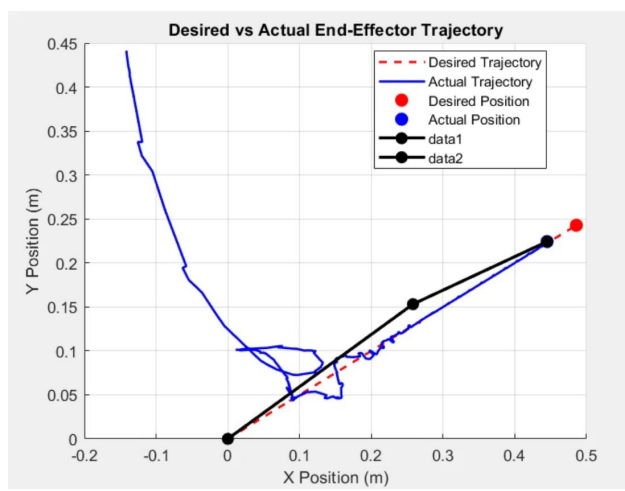

Figure 11: Trajectory tracking with $K_p = 1000$, $K_d = 80$ (oscillatory behavior).

6.3 Conclusions

The results demonstrated the critical impact of gain selection on the system's behavior:

- Smaller differences between K_p and K_d (e.g., Gain Set 1) resulted in a well-damped, smooth response, ideal for precise trajectory tracking.
- Larger discrepancies between K_p and K_d (e.g., Gain Set 2) led to underdamped behavior with oscillations and overshoot, indicating instability.
- Proper tuning of K_p and K_d is essential to balance stability and responsiveness, particularly for dynamic and arbitrary trajectories.

7 Compliance Control with Virtual Force Field

The aim of Task 9 is to design, implement, and test compliance control schemes for a two-link planar robot operating under simulated surgical conditions. The robot must precisely track desired

trajectories while responding to external forces and maintaining safe operation within constraints defined by a virtual wall. Compliance control allows the robot to adapt to external forces, such as those applied by a surgeon, ensuring smooth and flexible operation. At the same time, the virtual wall acts as a safety mechanism, restricting the robot's motion to within predefined boundaries by applying reactive forces when the end-effector approaches the limits. The integration of these two control aspects presents unique challenges. While compliance ensures adaptability, the virtual wall introduces sharp constraints that can destabilize the system if improperly managed. Careful tuning of control parameters and the inclusion of damping mechanisms are critical to maintaining stability and precision.

7.1 Control Design Overview

The compliance control law combines trajectory tracking, compliance to external forces, and virtual wall interactions:

$$\tau = J^T \left(K_p(x_d - x_a) + K_d(v_d - v_a) + K_s f_{\text{ext}} + f_{\text{wall}}\right) + g(q). \tag{20}$$

Here:

- K_p, K_d, K_s: Gains for position, velocity, and compliance control.
- x_d, v_d: Desired position and velocity; x_a, v_a: Actual position and velocity.
- f_{ext}: Simulated external forces (e.g., surgeon interaction).
- f_{wall}: Reactive forces from the virtual wall.
- $g(q)$: Gravity compensation.

The virtual wall applies forces proportional to boundary penetration:

$$f_{\text{wall}} = \begin{cases} K_{\text{wall}}(\text{boundary} - x_a), & \text{if out of bounds,} \\ 0, & \text{otherwise.} \end{cases} \tag{21}$$

The stiffness parameter K_{wall} governs the intensity of these forces.

7.2 Analysis and Observations

Free Space Behavior: In free space, the robot achieved high trajectory accuracy and responded smoothly to external forces due to the compliance control. Simulated surgeon forces resulted in minor, controlled deviations, demonstrating effective adaptation.

Near Virtual Wall Behavior: Approaching the virtual wall revealed instability characterized by oscillations and overshooting. High stiffness (K_{wall}) led to abrupt corrective forces, destabilizing the system near boundaries. These issues highlight the limitations of the current wall logic and the need for improved damping mechanisms.

Physical and Logical Insights:

- The coupling between compliance and wall forces creates complex dynamics near the boundaries, emphasizing the importance of gain tuning.
- Excessive stiffness amplifies oscillatory behavior, making precise control near constraints challenging.
- Reactive forces with sharp transitions are physically unrealistic and hinder smooth interactions.

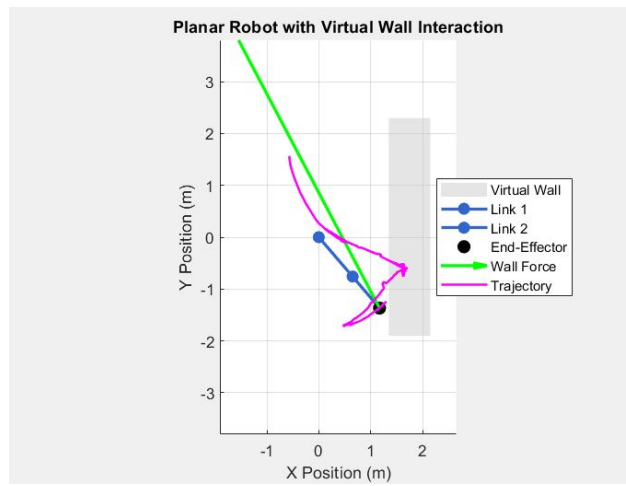

Figure 12: Robot trajectory and interaction forces near the virtual wall.

The compliance control scheme demonstrates significant promise in adapting to free-space forces and maintaining trajectory precision. However, the interaction with the virtual wall presents critical challenges, primarily due to high stiffness and insufficient damping in the wall logic. These limitations result in oscillations and overshooting, reducing the system's effectiveness in constrained environments.
To enhance performance, the integration of softer boundary responses, improved gain tuning, and damping mechanisms is essential. These adjustments will allow for smoother and safer interactions, aligning with the precision and reliability required in robotic-assisted surgical applications.

7.3 Improvements and Recommendations

To address these issues:

1. Tune K_p, K_d, and K_s to reduce oscillations and enhance damping near constraints.
2. Lower K_{wall} to provide smoother transitions when interacting with boundaries.
3. Introduce a damping term in the wall force logic to dissipate energy from abrupt corrections.
4. Use graded or nonlinear responses to boundary penetration for realistic and stable behavior.
5. Test under varying force profiles and wall conditions to ensure robustness across different scenarios.

8 Impedance Control Along a Diagonal Constraint

This section details the design, implementation, and testing of an impedance control scheme for a robotic system performing simulated eye surgery. The primary objective was to regulate the motion of the robot along a diagonal line constraint ($y = x$) while

ensuring adaptability to external forces. The system was evaluated for its trajectory tracking, dynamic stability, and interaction responsiveness.

8.1 Control Design and Implementation

The impedance control scheme governs the relationship between motion and applied forces through a well-defined control law. The motion-to-force relationship is described as:

$$F_{\text{imp}} = M_d\ddot{x}_d + D_d(\dot{x}_d - \dot{x}_a) + K_p(x_d - x_a) + F_{\text{ext}}, \tag{22}$$

where:

- M_d, D_d, K_p: Desired inertia, damping, and stiffness matrices.
- $x_d, \dot{x}_d, \ddot{x}_d$: Desired position, velocity, and acceleration.
- $x_a, \dot{x}_a$: Actual position and velocity.
- F_{ext}: External Cartesian forces simulating surgeon interaction.

To ensure the robot remains constrained to the diagonal ($y = x$), the desired motion and force projection were aligned along this constraint. Forces and torques were computed in Cartesian space and converted to joint torques using the Jacobian:

$$\tau = J^T F_{\text{imp}}, \tag{23}$$

where J is the Jacobian of the end-effector.

For enhanced control flexibility, two additional approaches were considered:

1. **Constraint-Based Control:** The diagonal constraint was explicitly enforced by setting:

$$x_a = [x; x], \quad \dot{x}_a = [\dot{x}; \dot{x}].$$

2. **Hybrid Force-Motion Control:** Impedance control along the diagonal was combined with a position control scheme to ensure adherence to the constraint:

$$\tau = J^T \left(F_{\text{imp}} + F_{\text{constraint}}\right). \tag{24}$$

The final control law was implemented as:

$$\tau = J^T \left(M_d\ddot{x}_d + D_d(\dot{x}_d - \dot{x}_a) + K_p(x_d - x_a) + F_{\text{ext}}\right) + G(q) + C(q, \dot{q})\dot{q}, \tag{25}$$

where:

- $G(q)$: Gravity compensation.
- $C(q, \dot{q})$: Coriolis and centrifugal forces.

8.2 Simulation and Testing

The control scheme was tested under three key scenarios:

1. **Free Motion:** The robot followed a diagonal trajectory without disturbances, showcasing high accuracy and stability.
2. **Force Interaction:** External forces applied at the end-effector simulated surgeon interactions, with the robot maintaining smooth constraint adherence.
3. **Boundary Interaction:** Virtual walls simulated workspace limits, where the robot exhibited stable and controlled behavior.

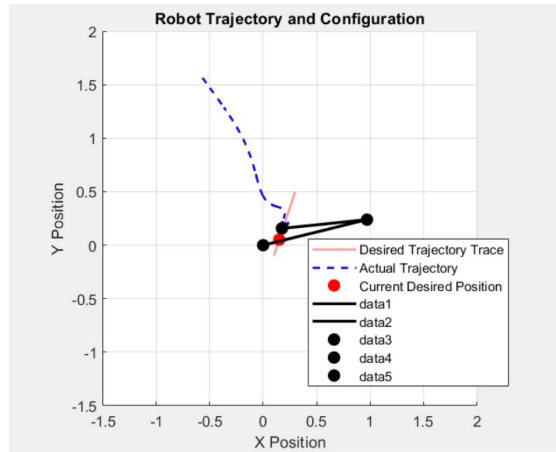

Figure 13: Robot trajectory and configuration during simulation. The desired trajectory trace (light red) and the actual trajectory (blue dashed line) are shown. The robot maintained stability and tracked the diagonal constraint effectively.

8.3 Results and Conclusions

The robot successfully tracked the diagonal trajectory ($y = x$) with minor deviations observed during free motion. Adaptive gain adjustments played a critical role in improving tracking accuracy by dynamically increasing stiffness and damping in response to larger position and velocity errors. Under external forces applied to simulate surgeon interactions, the system exhibited smooth compliance, maintaining the diagonal constraint with slight deviations that were quickly corrected. Near workspace boundaries defined by virtual walls, the robot behaved stably without oscillations, though higher stiffness settings led to abrupt corrections. This highlights the need for further refinement, such as implementing nonlinear damping strategies for smoother boundary behavior.

Overall, the impedance control scheme effectively regulated the robot's motion along the diagonal constraint while adapting to external forces. The system demonstrated high tracking precision, predictable compliance, and stable boundary interactions, making it well-suited for robotic-assisted eye surgery. Future improvements could focus on refining boundary behavior, optimizing external force compensation, and validating the scheme in real-world surgical scenarios.

9 Admittance Control

The primary goal of admittance control is to ensure the robot can adapt to disturbances while maintaining accurate tracking of a desired path, such as an elliptical trajectory. The control scheme implemented in this project integrates admittance dynamics with the robot's kinematics and dynamics. However, the initial trials exposed several critical challenges that required iterative refinements.

9.1 Admittance Control Law and Parameters

The admittance control law governs the interaction between the robot's motion and external forces. The Cartesian force (F_{imp}) applied at the end-effector is described by:

$$F_{\text{imp}} = M_d\ddot{x}_d + D_d(\dot{x}_d - \dot{x}_a) + K_p(x_d - x_a) + F_{\text{ext}}, \tag{26}$$

where:

- M_d: Desired inertia matrix,
- D_d: Desired damping matrix,
- K_p: Desired stiffness matrix,
- $x_d, \dot{x}_d, \ddot{x}_d$: Desired position, velocity, and acceleration,
- $x_a, \dot{x}_a$: Actual position and velocity,
- F_{ext}: External force applied at the end-effector.

The resulting joint torques (τ) are computed as:

$$\tau = J^T F_{\text{imp}} + G(q) + C(q, \dot{q})\dot{q}, \tag{27}$$

where:

- J: Jacobian matrix mapping joint velocities to end-effector velocities,
- $G(q)$: Gravity compensation torque vector,
- $C(q, \dot{q})$: Coriolis and centrifugal forces matrix,
- $q, \dot{q}$: Joint angles and velocities.

In the final implementation, the desired trajectory (x_d, y_d) is defined as an elliptical path:

$$x_d(t) = 0.5 + 0.2\sin(2\pi f t), \tag{28}$$
$$y_d(t) = 0.3 + 0.1\cos(2\pi f t), \tag{29}$$

where $f = 0.2$ Hz represents the frequency of the trajectory.

9.2 Challenges in Initial Implementation

The first implementation of the admittance control scheme revealed significant limitations. The robot exhibited severe oscillatory behavior, which caused the end-effector to deviate substantially from the desired trajectory, as shown in Figure 14. This instability was particularly pronounced during high-curvature segments of the trajectory, where the system struggled to adapt to rapid directional changes. Additionally, the robot lagged behind the desired trajectory, highlighting delays in response to the prescribed path.

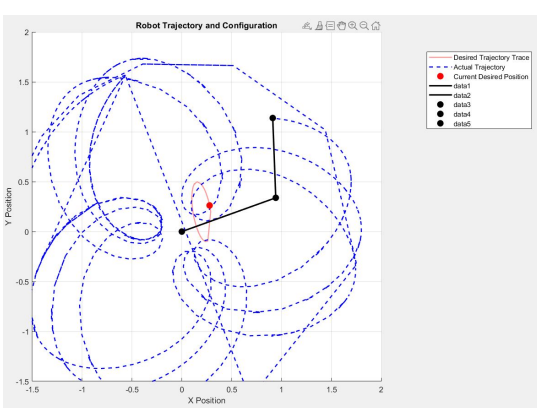

Figure 14: Initial results showing significant oscillations and deviation from the desired trajectory.

Another notable challenge was the sensitivity of the system to parameter tuning. Small adjustments to key parameters, such as stiffness (K_p) and damping (D_d), often resulted in disproportionately large deviations, indicating that the initial parameter values were poorly calibrated. These issues not only hindered the robot's ability to track the desired trajectory accurately but also amplified instabilities, leading to erratic motion patterns.

9.3 Refinements and Transition to Python Implementation

Despite efforts to refine the control law within the MATLAB/Simulink framework, the results remained unsatisfactory. The system's sensitivity to parameter changes and the lack of flexibility in the simulation environment limited progress. This prompted a transition to Python, where a more modular and adaptable implementation was developed. Python's flexibility allowed for advanced trajectory smoothing techniques and precise tuning of control parameters, which were critical in addressing the instability observed during earlier trials.

One of the most effective refinements was the introduction of trajectory smoothing. The desired position (x_d) was filtered using a moving average to prevent abrupt changes, calculated as:

$$x_d^{\text{smooth}} = \alpha x_d + (1 - \alpha)x_d^{\text{prev}}, \tag{30}$$

where α is a smoothing factor (typically $\alpha = 0.8$), and x_d^{prev} is the previous desired position. This approach significantly reduced oscillations and ensured a more stable interaction between the robot and its environment. The final Python-based simulation demonstrated the successful implementation of admittance control, achieving precise tracking of the desired elliptical trajectory with minimal deviations, as depicted in Figure 16. The system also exhibited a smooth and compliant response to external forces, ensuring stable operation under dynamic conditions.

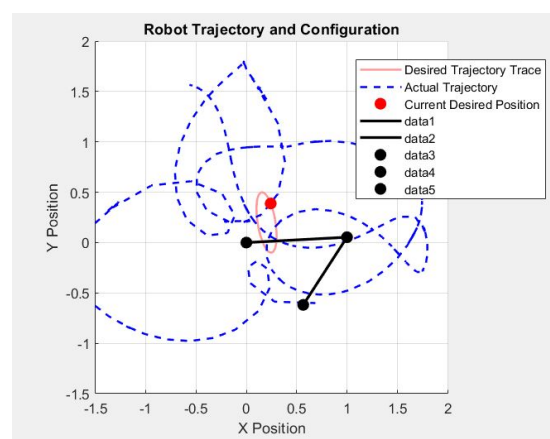

Figure 15: Intermediate refinements showing reduced oscillations but persistent trajectory delays.

9.4 Results and Observations

The progression from the initial trials to the final implementation highlights the importance of parameter tuning and trajectory smoothing in achieving stable and accurate admittance control. The initial oscillatory behavior and deviations, as shown in Figure 14, were effectively mitigated in the Python-based implementation. The refined control law enabled the robot to achieve con-

sistent tracking of the desired trajectory, as illustrated in Figures 16 and 15.

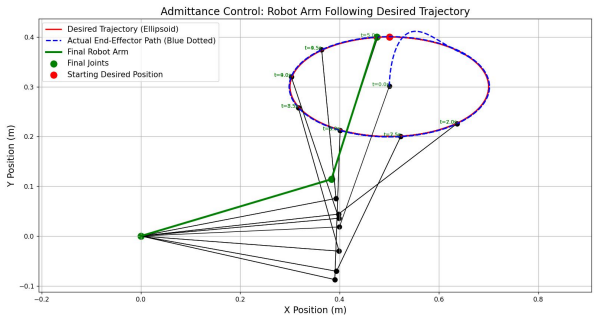

Figure 16: Final Python-based implementation demonstrating precise trajectory tracking and stable operation.

The Python-based implementation not only achieved the desired tracking performance but also provided a robust framework for future enhancements, including adaptive gain tuning and dynamic obstacle avoidance.

10 Three-Link Planar Robot with Obstacle Avoidance

In this task, a joint trajectory for a 3-link planar robot moving along a straight-line trajectory while avoiding a vital area was designed, tested, and visualized. The objective was to ensure that the robot avoids collisions with the vital area while maintaining its straight-line motion. The robot consists of three links with lengths $l_1 = 1\,\text{m}$, $l_2 = 1\,\text{m}$, and $l_3 = 0.3\,\text{m}$. The main challenges were designing the trajectory generation algorithm and ensuring proper obstacle avoidance.

10.1 Trajectory Generation Logic

The trajectory is generated for the robot's end-effector by moving from the initial position $(x_{\text{start}}, y_{\text{start}})$ to the final position $(x_{\text{end}}, y_{\text{end}})$ along a straight line. The motion is executed over a total simulation time t_{total}, which is normalized to a progression factor that updates the robot's position at each time step.

The key challenge was ensuring that the trajectory does not intersect with a "vital area" or obstacle, defined as a circular region with a given center $(x_{\text{vital}}, y_{\text{vital}})$ and radius r_{vital}. When the robot's end-effector comes too close to the vital area, it alters its trajectory to avoid collision. The avoidance strategy ensures that the robot steers around the vital area, adjusting its path without deviating too much from the original straight-line trajectory.

The following functions were implemented:

- **Inverse Kinematics (InverseKinematics.m)**: This function computes the joint angles $[q_1, q_2, q_3]$ for the robot given a Cartesian target position $[x, y]$. The robot avoids the vital area by adjusting the target position if it falls within the sensor radius. The function calculates the joint angles using standard inverse kinematics formulas, ensuring the robot can reach the desired target while avoiding the obstacle. The robot's links are checked for proximity to the vital area, and if necessary, the robot's entire arm is moved back to avoid collision.
- **Forward Kinematics (ForwardKinematics3.m)**: This function computes the Cartesian position of the end-effector based on the given joint angles $[q_1, q_2, q_3]$. It uses the link lengths to calculate the position in 2D space. This is essential for visualizing the robot's configuration and ensuring that the robot moves according to the desired joint angles.
- **Trajectory Generation (trajectoryGenerator3_ 1.m)**: The trajectory is generated by first computing a straight-line path from the starting position to the target position. If the robot comes within the sensor radius of the vital area, the trajectory is adjusted by calculating the angle to the center of the vital area. The robot's trajectory is steered around the obstacle by increasing the distance from the vital area in the direction away from the obstacle. The detour is dynamically adjusted based on the proximity of the robot to the vital area, ensuring the robot avoids the obstacle without deviating too much from the original path.
- **Visualization (DisplayRobot3.m)**: This function visualizes the robot's configuration and its motion. It plots the robot's three links and the end-effector, ensuring the robot's movement and its avoidance of the vital area are clearly shown. The robot's joints are displayed, and the vital area is visualized as a circular region. If the end-effector is inside the vital area, the function adjusts the robot's position and shows it on the edge of the vital area.

10.2 Obstacle Avoidance Strategy

The core idea behind the obstacle avoidance is simple yet effective. The robot's end-effector is continuously monitored for its proximity to the vital area. If the end-effector comes too close (i.e., within the sensor radius), the trajectory is adjusted by moving the target along the line connecting the current position and the center of the vital area, making sure the robot stays a safe distance away.

The robot's avoidance strategy is reactive and depends on its proximity to the obstacle. The closer the robot gets to the vital area, the larger the detour becomes, ensuring the robot steers clear of the obstacle. The code implements a smooth transition for the trajectory, allowing the robot to maintain a safe path while avoiding the vital area.

10.3 Visualization of the Robot's Motion

While the logic for generating the trajectory and avoiding the obstacle was successfully implemented, the visualization did not work as planned. The primary challenge with visualization was the difficulty in displaying a smooth transition in the robot's motion when avoiding the vital area, as the avoidance adjustment caused the robot's path to appear somewhat erratic at times. The adjustment to the trajectory, while functional, could not fully reflect a continuous, smooth movement for the robot's links.

The robot's configuration was visualized at each time step using the `DisplayRobot3` function, but the resulting plots showed sudden adjustments rather than smooth, continuous avoidance of the obstacle. This can be attributed to the simple detour method, which caused abrupt changes in the trajectory when the robot was close to the vital area.

To improve this, a smoother path-planning method could be implemented. For example, using a spline-based approach or Bézier curves would allow for continuous and smoother avoidance trajectories. Additionally, optimizing the trajectory with a path-planning

algorithm, such as A* or RRT (Rapidly-exploring Random Tree), could further enhance the obstacle avoidance mechanism.

10.4 Conclusion

The design and implementation of the robot's trajectory involved the integration of inverse kinematics, forward kinematics, and a trajectory generator that adjusts the robot's path to avoid obstacles. The approach is simple yet effective for basic obstacle avoidance, ensuring the robot stays within a safe region. The robot's motion is visualized to provide clear insights into how the avoidance mechanism works.

Although the current method provides an adequate solution for basic obstacle avoidance, the visualization could be improved for smoother transitions and more complex motion planning. Future work could focus on optimizing the trajectory generation using smoother path-planning algorithms and refining the visualization to reflect continuous and natural movement for the robot. Additionally, more advanced techniques could be applied to handle dynamic obstacles or real-time adjustments during robot motion.